

TITANIC DATA ANALYSIS PROJECT

Internship Report

Full Name: Armish Shabbir

Internship Domain: Data Analysis

Email Address: armishkismis@gmail.com

Phone Number: +923093725474

Submitted To: Arch Technologies

Month: Month 1

Prepared as part of internship submission requirements

1. Introduction

This project focuses on analyzing the Titanic dataset to understand the key factors that influence passenger survival. The objective is to clean the dataset, perform exploratory data analysis (EDA), and extract meaningful insights using Python-based data analysis tools.

2. Dataset Description

The dataset contains detailed information about Titanic passengers, including:

- Age
- Gender
- Passenger Class (Pclass)
- Fare
- Embarked Port
- Survival Status

Source: Kaggle Titanic Dataset

3. Initial Data Exploration

Before performing any data cleaning, an initial exploration of the dataset was conducted to understand its structure and identify potential issues.

Code:

```
import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv("titanic/train.csv")

print("=== RAW DATA OVERVIEW ===")
print(f"Shape: {df.shape}")
print("\nColumns:")
```

```

print(df.columns.tolist())
print("\nData types:")
print(df.dtypes)
print("\nMissing values:")
print(df.isnull().sum())
print("\nFirst 5 rows:")
df.head()

```

Explanation:

This step provides an overview of the dataset, including its size, structure, and missing values. It helps identify data quality issues before applying cleaning techniques.

4. Data Cleaning and Feature Engineering

The dataset was cleaned and transformed to improve its quality and prepare it for analysis.

Code:

```

# Create a copy of dataset
df_clean = df.copy()

# Drop unnecessary columns
cols_to_drop = ['PassengerId', 'Ticket', 'Cabin', 'Name']
df_clean.drop(columns=cols_to_drop, inplace=True)

# Extract Title from Name
df['Title'] = df['Name'].str.extract(' ([A-Za-z]+)\.', expand=False)

# Map titles
title_map = {
    'Mr': 'Mr', 'Miss': 'Miss', 'Mrs': 'Mrs', 'Master': 'Master',
    'Dr': 'Officer', 'Rev': 'Officer', 'Col': 'Officer', 'Major':
    'Officer',
    'Capt': 'Officer', 'Jonkheer': 'Royalty', 'Don': 'Royalty',
    'Sir': 'Royalty', 'Lady': 'Royalty', 'Countess': 'Royalty',
    'Dona': 'Royalty', 'Ms': 'Miss', 'Mme': 'Mrs', 'Mlle': 'Miss'
}

```

```
df['Title'] = df['Title'].map(title_map).fillna('Rare')
df_clean['Title'] = df['Title']

# Convert categorical columns
cat_cols = ['Pclass', 'Sex', 'Embarked', 'Title']
df_clean[cat_cols] = df_clean[cat_cols].astype('category')

# Handle missing values
df_clean['Age'] = df_clean.groupby(['Sex',
    'Pclass'])['Age'].transform(
    lambda x: x.fillna(x.median())
)

df_clean['Embarked'] =
df_clean['Embarked'].fillna(df_clean['Embarked'].mode()[0])

# Fill Fare if needed
df_clean['Fare'] = df_clean.groupby('Pclass')['Fare'].transform(
    lambda x: x.fillna(x.median())
)

# Feature engineering
df_clean['FamilySize'] = df_clean['SibSp'] + df_clean['Parch'] + 1
df_clean['IsAlone'] = (df_clean['FamilySize'] == 1).astype(int)

# Final check
print(df_clean.info())
df_clean.head()
```

Explanation:

- Removed irrelevant columns to simplify the dataset
- Extracted the "Title" feature to capture social status
- Converted categorical columns for better processing
- Handled missing values using appropriate statistical methods
- Created new features (Family Size and Is Alone) to enhance analysis

This step significantly improved data quality and prepared the dataset for accurate analysis.

5. Data Analysis (EDA)

Exploratory Data Analysis was conducted to identify patterns and relationships within the dataset.

Key analyses performed:

- Survival rate based on gender
- Survival rate based on passenger class
- Impact of family size on survival

6. Visualizations

The following visualizations were created to better understand survival patterns in the dataset.

Code:

```
import matplotlib.pyplot as plt
import numpy as np

# Work on a copy
data = df_clean.copy()

# Helper function
def survival_rate(group_col):
    return data.groupby(group_col)['Survived'].mean()

surv_by_sex = survival_rate('Sex')
surv_by_class = survival_rate('Pclass')
ages = data['Age'].dropna()
surv_by_famsize = survival_rate('FamilySize')

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle('Titanic Dataset - Survival Insights', fontsize=16)

axes[0, 0].bar(surv_by_sex.index, surv_by_sex.values)
axes[0, 0].set_title('Survival Rate by Gender')

axes[0, 1].bar(surv_by_class.index, surv_by_class.values)
axes[0, 1].set_title('Survival Rate by Passenger Class')

axes[1, 0].hist(ages, bins=30)
axes[1, 0].set_title('Age Distribution')

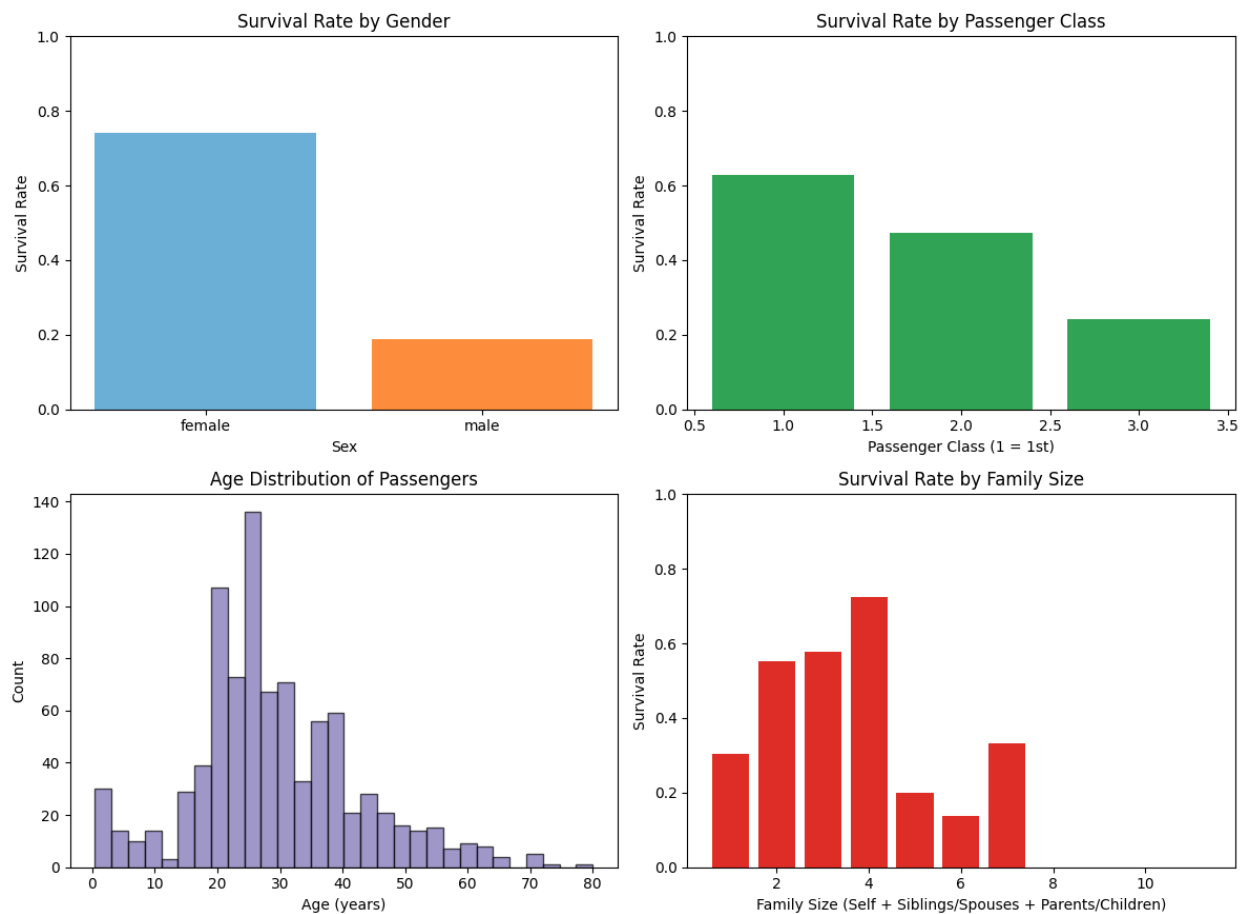
axes[1, 1].bar(surv_by_famsize.index, surv_by_famsize.values)
axes[1, 1].set_title('Survival Rate by Family Size')

plt.tight_layout()
plt.show()
```

Explanation:

- A 2x2 subplot layout was used to visualize multiple insights in one figure
- Bar charts were used for categorical comparisons (gender, class, family size)
- Histogram was used to analyze age distribution
- These visualizations help in identifying patterns and trends in passenger survival

Titanic Dataset – Survival Insights



7. Key Insights

- Female passengers had significantly higher survival rates
- First-class passengers had better chances of survival
- Family size influenced survival probability
- Socio-economic factors played a major role in survival outcomes

8. Conclusion

The analysis highlights that gender, passenger class, and social factors were key determinants of survival in the Titanic disaster. The project demonstrates how data cleaning and analysis can uncover meaningful insights from raw datasets.

9. Tools Used

- Python
- Pandas
- NumPy
- Matplotlib
- VS Code

10. Screenshots

Dataset Preview

Loading Titanic dataset and displaying first few rows using pandas (`df.head()`) to understand structure

```
Import pandas as pd

df = pd.read_csv("titanic/train.csv")
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cummings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikinen, Miss. Laina	female	26.0	0	0	STON/O2, 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Dataset Information

Checking dataset structure, data types, and missing values using `df.info()`.

```
import pandas as pd
import numpy as np

df = pd.read_csv(r"titanic/train.csv")

print("=== RAW DATA OVERVIEW ===")
print(f"Shape: {df.shape}")
print("\nColumns:")
print(df.columns.tolist())
print("\nData types:")
print(df.dtypes)
print("\nMissing values:")
print(df.isnull().sum())
print("\nFirst 5 rows:")
display(df.head())
```

[2]

```
'''
=== RAW DATA OVERVIEW ===
Shape: (891, 12)

Columns:
['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']

Data types:
PassengerId    int64
Survived        int64
Pclass          int64
Name            str
Sex             str
Age            float64
SibSp           int64
Parch           int64
Ticket          str
Fare            float64
Cabin           str
Embarked        str
dtype: object

Missing values:
PassengerId    0
Survived        0
Pclass          0
...
Embarked        2
dtype: int64

First 5 rows:
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Data Cleaning Step

Handling missing values and dropping irrelevant columns to prepare a clean dataset for analysis.

```
# -----
# 2 Clean the dataset
# -----
df_clean = df.copy() # work on a copy to keep raw data intact

# 3.1 Drop columns that are IDs or too noisy
cols_to_drop = ['PassengerId', 'Ticket', 'Cabin', 'Name']
df_clean.drop(columns=cols_to_drop, inplace=True)
# Reason: PassengerId & Ticket add no predictive signal.
# Cabin is >75% missing - extracting deck would be noisy.
# Name will be replaced by a more informative Title feature.

# 3.2 Extract Title from Name (useful surrogate)
# We need the original Name column for this step, so we re-extract it:
df['Title'] = df['Name'].str.extract('([A-Za-z]+)\.', expand=False)
# Map rare titles to a common group
title_map = {
    'Mr': 'Mr', 'Miss': 'Miss', 'Mrs': 'Mrs', 'Master': 'Master',
    'Dr': 'Officer', 'Rev': 'Officer', 'Col': 'Officer', 'Major': 'Officer',
    'Capt': 'Officer', 'Jonkheer': 'Royalty', 'Don': 'Royalty',
    'Sir': 'Royalty', 'Lady': 'Royalty', 'Countess': 'Royalty',
    'Dona': 'Royalty', 'Ms': 'Miss', 'Mme': 'Mrs', 'Mlle': 'Miss'
}
df['Title'] = df['Title'].map(title_map).fillna('Rare')
# Add Title to cleaned set and drop the raw Name column (already dropped)
df_clean['Title'] = df['Title']

# 3.3 Fix data types
cat_cols = ['Pclass', 'Sex', 'Embarked', 'Title']
df_clean[cat_cols] = df_clean[cat_cols].astype('category')
# Reason: reduces memory, signals to algorithms that these are nominal/ordinal.

# 3.4 Handle missing values
# Age - median per Sex & Pclass (more granular than global median)
df_clean['Age'] = df_clean.groupby(['Sex', 'Pclass'])['Age'].transform(
    lambda x: x.fillna(x.median())
)
# Embarked - mode (most common port)
df_clean['Embarked'] = df_clean['Embarked'].fillna(df_clean['Embarked'].mode()[0])
# Fare - if any missing (rare), fill with median of its Pclass
df_clean['Fare'] = df_clean.groupby('Pclass')['Fare'].transform(
    lambda x: x.fillna(x.median())
)

# 3.5 Optional feature engineering (helps EDA & modeling)
df_clean['FamilySize'] = df_clean['SibSp'] + df_clean['Parch'] + 1
df_clean['IsAlone'] = (df_clean['FamilySize'] == 1).astype(int)

# 3.6 Final check
print("\n== CLEANED DATA OVERVIEW ==")
print(f"Shape: {df_clean.shape}")
print("\nData types:")
print(df_clean.dtypes)
print("\nMissing values after cleaning:")
print(df_clean.isnull().sum())
print("\nFirst 5 rows:")
display(df_clean.head())
```

Save the Cleaned Dataset

```
=== CLEANED DATA OVERVIEW ===
Shape: (891, 11)

Data types:
Survived      int64
Pclass        category
Sex            category
Age           float64
SibSp         int64
Parch         int64
Fare          float64
Embarked       category
Title          category
FamilySize    int64
IsAlone       int64
dtype: object

Missing values after cleaning:
Survived      0
Pclass        0
Sex            0
Age           0
SibSp         0
Parch         0
...
IsAlone       0
dtype: int64

First 5 rows:
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings.
C:\Users\Public\Documents\WonderShare\Creator\Temp\ipykernel_9992\1851255874.py:15: SyntaxWarning: "\." is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\."? A raw string is also an option.
C:\Users\Public\Documents\WonderShare\Creator\Temp\ipykernel_9992\1851255874.py:15: SyntaxWarning: "\." is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\."? A raw string is also an option.
df['Title'] = df['Name'].str.extract('([A-Za-z])\.', expand=False)

...


|   | Survived | Pclass | Sex    | Age  | SibSp | Parch | Fare    | Embarked | Title | FamilySize | IsAlone |
|---|----------|--------|--------|------|-------|-------|---------|----------|-------|------------|---------|
| 0 | 0        | 3      | male   | 22.0 | 1     | 0     | 7.2500  | S        | Mr    | 2          | 0       |
| 1 | 1        | 1      | female | 38.0 | 1     | 0     | 71.2833 | C        | Mrs   | 2          | 0       |
| 2 | 1        | 3      | female | 26.0 | 0     | 0     | 7.9250  | S        | Miss  | 1          | 1       |
| 3 | 1        | 1      | female | 35.0 | 1     | 0     | 53.1000 | S        | Mrs   | 2          | 0       |
| 4 | 0        | 3      | male   | 35.0 | 0     | 0     | 8.0500  | S        | Mr    | 1          | 1       |



...

# -----
# Export cleaned data
# -----
output_path = "titanic_cleaned.csv"
df_clean.to_csv(output_path, index=False)
print(f"Cleaned data saved to: {output_path}")

(4)
Cleaned data saved to: titanic_cleaned.csv
```

End of Report